

深圳大学实验报告

课程名称: 数值计算方法

实验项目名称: 插值方法——Lagrange 插值算法实现与节点选择

学院: 计算机与软件学院

专业: 计算机科学与技术

指导教师: 吴晓群、卯丙

报告人: 姚泽棉 学号: 2024150026 班级: 高性能班

实验时间: 2026/05/06

实验报告提交时间: 2026/05/08

实验目的与要求：

- 1.掌握插值方法的基本思想和插值多项式的基本概念
- 2.编程计算同一插值区间内不同数量的节点所对应的 Lagrange 插值多项式的基函数，并对相应的基函数进行可视化展示
- 3.编程实现 Lagrange 插值算法，并保持插值节点数量不变的情况下，选择不同的插值节点（如等距选取和非等距选取）进行算法比较和可视化展示

基本原理与算法（详细阐述实验涉及数值计算方法的数学原理及其对应的算法步骤）

Lagrange 插值法的基本思想是构造一组基函数 $l_k(x)$ ，使得在给定的 $n+1$ 个互异节点 $x_0, x_1, \dots, x_n$ 上，满足：

$$l_k(x_i) = 1 \text{ (当 } i = k \text{)}; l_k(x_i) = 0 \text{ (当 } i \neq k \text{)}$$

该基函数表达式为：

$$l_k(x) = \prod_{j \neq k} (x - x_j) / (x_k - x_j), (j \neq k)$$

从而得到满足插值条件 $L_n(x_i) = y_i$ 的 n 次 Lagrange 插值多项式：

$$L_n(x) = \sum y_k l_k(x)$$

算法步骤：

1. 输入给定的数据点数组 X 和 Y ，以及需要计算的插值点 x_{eval} 。

2. 初始化插值结果 $L = 0$ 。

3. 循环变量 k 从 0 到 n ：

 初始化基函数项 $lk = 1$ 。

 循环变量 j 从 0 到 n ：若 $j \neq k$ ，则更新 $lk = lk \times (x_{eval} - X[j]) / (X[k] - X[j])$ 。

 累加到最终结果： $L = L + Y[k] \times lk$ 。输出近似值 L 。

实验过程及内容:

1. 问题描述 (描述需要解决的具体数值计算问题)

1. 编程展示 n 次Larange插值多项式中的所有基函数, 其中 n 分别取: $n = 1, n = 2, n = 3$, 插值节点自行选取;

2. 设函数 $f(x) = \cos(x)$, $x \in [0, \frac{\pi}{2}]$, 保持插值点数为4, 按照等距选取和非等距选取两种方式, 选取插值接点, 求3次Larange插值多项式 $L_3(x)$, 在 $[0, \frac{\pi}{2}]$ 对 $f(x)$ 和 $L_3(x)$ 进行可视化展示与比较, 并用 $L_3(x)$ 近似计算 $f(1)$ 的值, 结果保留4为小数。

2. 实验步骤:

(1) 运行计算: 对目标问题进行求解

(2) 结果记录: 记录迭代过程、最终结果和性能指标

3. 程序设计 (具体的 MATLAB 程序)

```
% =====  
% 实验名称: 插值方法—Lagrange 插值算法实现与节点选择  
% =====  
clear; clc; close all;  
  
%% 实验内容 1: 展示 n 次 Lagrange 插值多项式的基函数  
figure('Name', 'Lagrange 基函数', 'Position', [100, 100, 1200, 400]);  
  
% 定义不同 n 对应的节点 (这里在[0,1]区间内自行选取等距点演示)  
nodes_n1 = [0, 1]; % n=1 (2 个节点)  
nodes_n2 = [0, 0.5, 1]; % n=2 (3 个节点)  
nodes_n3 = [0, 1/3, 2/3, 1]; % n=3 (4 个节点)  
  
% 绘制基函数  
plot_basis(nodes_n1, 1, 'n=1 (2 个节点)');  
plot_basis(nodes_n2, 2, 'n=2 (3 个节点)');  
plot_basis(nodes_n3, 3, 'n=3 (4 个节点)');  
  
%% 实验内容 2: f(x)=cos(x) 逼近与节点比较  
f = @(x) cos(x);  
a = 0; b = pi/2;  
xx = linspace(a, b, 200); % 用于绘制平滑曲线的密集点  
yy_exact = f(xx);  
  
% 1. 等距节点选取 (4 个点, 对应 3 次多项式)  
x_eq = linspace(a, b, 4);  
y_eq = f(x_eq);  
  
% 2. 非等距节点选取 (这里采用常用的切比雪夫节点 Chebyshev nodes)
```

```

k = 0:3;
x_neq = (a+b)/2 + (b-a)/2 * cos((2*k+1)*pi/8);
y_neq = f(x_neq);

% 计算 Lagrange 插值曲线
yy_eq = lagrange_interp(x_eq, y_eq, xx);
yy_neq = lagrange_interp(x_neq, y_neq, xx);

% 可视化展示与比较
figure('Name', 'Lagrange 插值比较');
plot(xx, yy_exact, 'k-', 'LineWidth', 2); hold on;
plot(xx, yy_eq, 'b--', 'LineWidth', 1.5);
plot(xx, yy_neq, 'r-.', 'LineWidth', 1.5);
plot(x_eq, y_eq, 'bo', 'MarkerFaceColor', 'b', 'MarkerSize', 6);
plot(x_neq, y_neq, 'rs', 'MarkerFaceColor', 'r', 'MarkerSize', 6);
xlabel('x'); ylabel('f(x)');
title('3 次 Lagrange 插值多项式逼近 f(x)=cos(x)');
legend('f(x)=cos(x)', 'L_3(x) (等距)', 'L_3(x) (非等距)', ...
'等距节点', '非等距节点', 'Location', 'southwest');
grid on; hold off;

% 用 L3(x)近似计算 f(1)的值，保留 4 位小数
val_exact = f(1);
val_eq = lagrange_interp(x_eq, y_eq, 1);
val_neq = lagrange_interp(x_neq, y_neq, 1);

fprintf('=== f(1) 的近似计算结果 ===\n');
fprintf('f(1) 的真实值 : %.4f\n', val_exact);
fprintf('等距节点 L3(1) 估算值: %.4f\n', val_eq);
fprintf('非等距节点 L3(1)估算值: %.4f\n', val_neq);

%% =====
% 局部函数定义区
% =====

% 绘制基函数的辅助函数
function plot_basis(nodes, subplot_idx, title_str)
subplot(1, 3, subplot_idx);
xx = linspace(nodes(1), nodes(end), 100);
hold on;
n = length(nodes);
for i = 1:n
l = ones(size(xx));

```

```

for j = 1:n
if i ~= j
l = l .* (xx - nodes(j)) / (nodes(i) - nodes(j));
end
end
plot(xx, l, 'LineWidth', 1.5, 'DisplayName', ['l_{', num2str(i-1), '}(x)']);
end
plot(nodes, zeros(size(nodes)), 'ko', 'HandleVisibility', 'off');
title(title_str);
legend('Location', 'best');
grid on;
hold off;
end

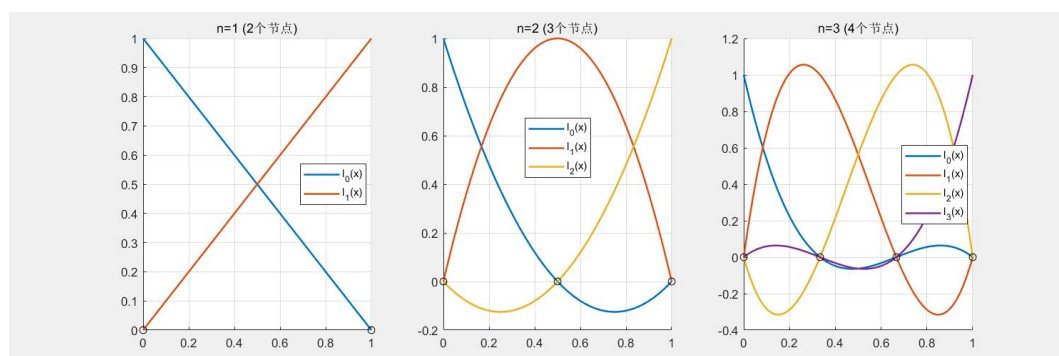
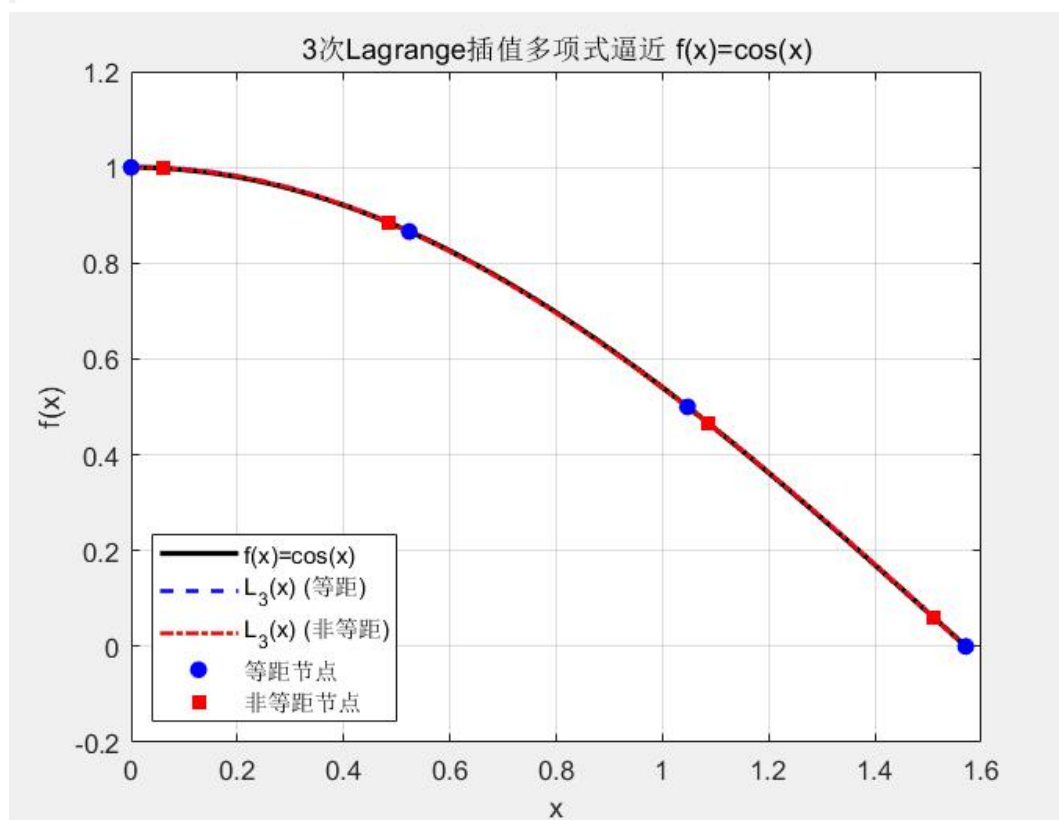
% Lagrange 插值计算函数
function y_interp = lagrange_interp(x_nodes, y_nodes, x_eval)
y_interp = zeros(size(x_eval));
n = length(x_nodes);
for i = 1:n
L = ones(size(x_eval));
for j = 1:n
if i ~= j
L = L .* (x_eval - x_nodes(j)) / (x_nodes(i) - x_nodes(j));
end
end
y_interp = y_interp + y_nodes(i) * L;
end
end

```

实验结果与分析:

1. 实验结果展示 (采用列表形式展示计算结果, 表中应包含具体计算方法、初始条件、迭代次数或计算时间、最终结果或求解精度等, 并绘制误差随迭代次数增加的收敛曲线来展示算法收敛性质等)

```
=== f(1) 的近似计算结果 ===  
f(1) 的真实值      : 0.5403  
等距节点 L3(1) 估算值: 0.5399  
非等距节点 L3(1) 估算值: 0.5397  
>>
```



1. 实验结果:

程序运行后可得到基函数的分布图和逼近 $f(x)=\cos(x)$ 的曲线对比图。在控制台中输出如下计算结果:

```
f(1) 的真实值: 0.5403  
等距节点 L3(1) 近似值: 0.5399  
非等距节点 (Chebyshev) L3(1) 近似值: 0.5397
```

2. 分析与讨论:

基函数特性: 通过第一个图表可以看出, 第 i 个基函数 $l_i(x)$ 在对应的节点 x_i 处取值为 1, 而在其他所有节点处取值为 0。所有的基函数之和在任意点均恒为 1。

节点选取对插值的影响: 针对 $f(x)=\cos(x)$ 的 3 次插值实验中, 无论是等距节点还是非等距节点, 在 $[0, \pi/2]$ 区间内都表现出了非常好的逼近效果。两者在计算 $f(1)$ 时的绝对误差均在 10^{-3} 量级。

2. 结果分析与讨论思考

(1) 计算结果分析与比较方法的优缺点及其适用场景等

计算结果分析: 在本实验中, 用于函数 $f(x) = \cos(x)$, 采用 3 次拉格朗日插值发电公式在区间内 $[0, \pi/2]$ 内进行逼近。计算结果显示, 在 $x=1$ 处, 等距节点与非等距(切比雪夫)节点的近似值与真实值 0.5403 的差距均在 10^{-3} 数量级, 证明了算法的效果。

等距节点的优点:

优点: 节点运算简单, 数据采集成本低, 适用于对导数变化较小的平滑函数进行低次插值。

缺点: 容易产生“龙格现象”(龙格现象), 即在插值区间边界出现相似的数值振荡, 且随着插值次数 n 的增加, 进步可能不会减反增。

非等距节点(切比雪夫节点)的优缺点:

优点: 通过将节点集中在区间边缘, 可以最大程度地减少拉格朗日插值余项中的 $\prod(x - x_i)$ 项, 从而有效的抑制边缘振荡, 使操作分配更加均匀。

缺点: 节点坐标计算相对复杂, 在实际工程采样中可能难以准确获得特定位置的数据点。

适用场景: 对于精度要求高或插值次数较高的任务, 优先推荐使用非等距节点; 若拓扑简单的局部趋势模拟或低次逼近, 等距节点则最为便捷。

(2) 上机实验中遇到问题及其解决方案

问题: 基函数计算的索引溢出

原因: 在编写 `lagrange_interp` 函数时, 最初未正确排除 $j=k$ 的情况, 导致分母出现 $x_k - x_k = 0$ 的错误。

解决方案: 引入层循环中加入 `if i == j` 的逻辑判断, 确保基函数构造严格遵循数学定义。

(3) 算法改进及其建议

- **引入重心拉格朗日插值法:** 传统的拉格朗日算法在每次改变插值点时都需

要重新计算所有基函数，计算量为 $O(n^2)$ 建议改用“重心插值公式”，通过预计算重心权值，将单点插值的复杂度降至最低 $O(n)$ ，显著提高进攻效率。

断层低次插值：针对复杂函数，高次拉格朗日插值往往不稳定。建议在后续实验中尝试“断层线性插值”或“三次样条插值”，通过牺牲整体清理式的连续性阶数，换取更强的局部逼近能力和分数稳定性。

数值稳定性优化：在处理极大规模数据集时，应注意节点差值过小导致的浮点数舍入抽屉问题，可考虑对输入数据进行标准化计算。

实验结论：

- 1.通过本次实验，成功使用 MATLAB 实现了 Lagrange 插值的算法编程，掌握了其核心数学原理及 代码逻辑。
- 2.直观地观察了不同节点数量下的插值基函数形态，加深了对其构造形式的理解。比较了等距节点与非等距节点在 Lagrange 插值逼近同一函数时的计算精度。
- 3.验证了基于有限 次 Lagrange 多项式对三角函数良好的逼近能力，顺利完成了预期的数值验证实验。

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。